

Guarantee D — Conflict States Are Preserved: Conflict-Preservation Determinism as a Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the five guarantees of the CKS determinism contract — **Guarantee D, conflict-preservation determinism** — as a standalone architectural commitment with independent operational content, separable from the four other guarantees with which it composes and from the cell-level resolution behavior it does not specify.

Abstract

The CKS determinism contract names five guarantees over substrate behavior (§4.1, §11.3 of the source paper). Prior decomposition notes formalize the contract's integrating frame, the substrate/model distinction that scopes it, and Guarantees A through C. This note formalizes Guarantee D — conflict-preservation determinism — as having independent architectural content separable from the other four. Guarantee D specifies that the substrate-level preservation of conflicts is itself deterministic: same substrate state produces same conflict reads; cell-level resolution decisions do not erase the conflict record; conflict provenance is stable across operations; conflict relationships remain stable across operations that do not explicitly modify them. The guarantee is what makes the conflict-as-first-class commitment operationally rigorous at the determinism level, and what makes the substrate's authority for "what is in conflict" operationally realizable. This note states the four operational components, distinguishes them from four adjacent conflict-handling patterns commonly conflated with them (eventual conflict resolution, last-write-wins, automatic infrastructure-layer reconciliation, conflict suppression), names the failure modes that violate the guarantee specifically, and provides an operational test for whether a given system implements it.

1. Why Guarantee D needs to be formalized as standalone

The parent foundational note A1.10 commits the CKS pattern to a determinism contract with five guarantees over substrate behavior. The integrating-frame note A2.55 enumerated the five at the architectural level; A2.56 formalized the substrate/model distinction that scopes them; A2.57, A2.58,

and A2.59 formalized Guarantees A, B, and C. This note formalizes Guarantee D — conflict-preservation determinism — with particular weight on making the conflict-as-first-class commitment from A1.03 operationally rigorous at the determinism level.

The motivating cases are deployments where conflict states must be deterministically preserved across reads and operations. A regression test verifying conflict-handling behavior across cell executions needs deterministic conflict reads or it cannot pass or fail the assertion stably. A governance review examining what conflicts existed at a specific time needs deterministic conflict reads or it may see different content depending on when or how it queries. An audit query asking whether a specific contradiction was preserved before and after a resolution needs the conflict record to be both addressable and non-erased by the resolution. Each requires Guarantee D — preservation must be deterministic, not merely nominal.

A second motivation is the strategic prior-art posture. Guarantee D forecloses architectures where conflict states are non-deterministic — varying across reads, erased on resolution, with varying provenance, or with unstable relationships. Patentable derivations focused on conflict-handling architectures with deterministic preservation, audit-coherent conflict tracking, or replayable conflict-state architectures are substantially more defensibly contested when Guarantee D is publicly formalized as standalone.

A third motivation is the connection to A1.03 and its decomposition. A1.03 commits to conflicts being preserved as first-class substrate state, with substrate-level preservation specified in A2.13, cell-level resolution in A2.14, and the two-level coupling in A2.15. Guarantee D is the determinism property that makes A2.13's preservation operationally rigorous and A2.14's resolution coherent with preservation. A2.13 alone admits implementations where conflicts are preserved nominally but vary across reads or are erased by resolution. Guarantee D rules those out.

2. The Guarantee D commitment, defined precisely

A substrate satisfies Guarantee D if and only if all four of the following components hold throughout the substrate's existence.

(a) Deterministic conflict reads. For any substrate state *S* containing preserved conflict states per A2.13, reads against *S* return the same conflict states deterministically. Conflict reads inherit Guarantee A's read-determinism (A2.57): same substrate state produces same content reads. Guarantee D adds, on top of that, a content-class commitment — that the substrate content called "conflict state" is itself deterministic, including the contradicting content, the relationship metadata per A2.16, and the conflict provenance per A2.40. Without component (a), the content class itself could be silently reduced by erasure-on-resolution and Guarantee A's general read-determinism would not detect the loss.

(b) Resolution-without-erasure. When a cell executes a resolution decision per A2.14, the resolution is recorded as additional substrate content per Guarantee C's addressability (A2.59), but the original conflict state per A2.13 is not erased. The substrate after resolution contains both the original conflict and the resolution decision, both addressable. Resolution operations do not modify the conflict state in place; they add resolution content alongside the preserved conflict. Component (b) is what keeps

cell-level resolution per A2.14 coherent with substrate-level preservation per A2.13.

(c) Deterministic conflict provenance. Provenance attached to conflict states is deterministic across operations. The four conflict-specific provenance fields per A2.16 (writer attribution for each contradicting position, timestamp for each, rationale where applicable, relationship metadata between contradicting content) are stable across reads; the six broader provenance fields per A2.40 that all substrate content carries are also stable. Component (c) is what makes path retraceability per A1.07 operate over conflict states.

(d) Stable conflict relationships. Relationships between contradicting content — the bidirectional relationship metadata per A2.16, the OIDA-inherited signed-contradiction-edge construction per A2.17 — are stable across operations that do not explicitly modify them. Reads return consistent metadata regardless of which side of the contradiction is queried, which access path is used, or which time within the conflict's persistence interval the read occurs. A relationship that varies across reads is not a first-class addressable object architecturally; component (d) is what keeps A2.16's relationship-as-substrate-content commitment operationally testable.

The four components together define Guarantee D. Failing any one — even with the other three robustly satisfied — fails the guarantee.

3. What Guarantee D does NOT claim

The standalone treatment is not maximalist. Stating precisely what the guarantee does not require keeps the framing from drifting beyond what the source paper supports.

It does not claim resolution behavior is bit-deterministic. Cell behavior under resolution rules is governed by Guarantee B per A2.58 — same substrate state with same orchestration rules produces rule-equivalent cell behavior, not bit-identical output. Guarantee D specifies preservation determinism, not resolution determinism.

It does not require all conflicts to be preserved indefinitely. Deployments may have retention policies; the architectural commitment holds for the conflict's substrate residence. What happens after retention expiration is a deployment policy under the substrate's normal write authority per A1.01.

It does not foreclose deployment-layer conflict visualization. Adjacent components per Pattern B of A1.16 may produce derivative views with their own freshness properties. Guarantee D applies to the substrate's own conflict reads through standard substrate access paths per A2.25 Requirement 2, not to derivative views.

It does not specify implementation patterns. Implementations may use immutable conflict records, append-only conflict logs, versioned conflict storage, or other mechanisms. The architectural commitment is to the four components; the choice of mechanism is a deployment decision.

It does not require resolution decisions to be operationally authoritative immediately. The commitment is that resolution is recorded as substrate content per Guarantee C; how subsequent operations treat resolved conflicts is governed by orchestration rules per A2.14.

It does not require global cross-scope consistency. Conflicts are scope-specific; relationships between contradicting content are scope-specific. Component (d) commits to relationship stability within each scope, not across scopes.

4. What Guarantee D is NOT

Four adjacent conflict-handling patterns are commonly conflated with Guarantee D. Each is a reasonable commitment in some other architecture; naming the distinction prevents the misreading.

Not eventual conflict resolution. Eventual resolution is the distributed-systems pattern where conflicts arising from concurrent updates are eventually reconciled by infrastructure (last-write-wins, vector-clock resolution, CRDT-based merging). Reconciliation typically erases the conflict by replacing contradicting positions with a single reconciled value. Guarantee D is different: conflicts per A1.03 are architecturally preserved as first-class state. Resolution in the CKS sense is a cell-level operation under human-authored orchestration rules per A2.14, recorded as additional substrate content; reconciliation in the eventual-consistency sense is an infrastructure-layer overwrite that component (b) forbids.

Not last-write-wins (LWW). LWW is the concurrency-control pattern where the most recent write supersedes earlier writes with no record of the superseded content. LWW is operationally useful in distributed-systems scenarios; it is simply a different commitment. Under LWW the substrate carries only the latest position; under Guarantee D both contradicting positions and their relationship are preserved. LWW fails components (b) and (d).

Not automatic reconciliation. Automatic reconciliation runs conflict-resolution logic at infrastructure layer — replication conflict resolvers, distributed-systems consistency mechanisms, automated merge logic. Guarantee D is different: conflict preservation is at substrate layer per A2.13, resolution at cell layer per A2.14 under human-authored rules. The distinguishing question is who decides the reconciliation logic; for Guarantee D the answer must be humans, not infrastructure.

Not conflict suppression. Conflict suppression filters conflicts out of reads or transforms them into non-conflicting content before reaching readers — "clean" views that hide conflicts to "improve user experience." Guarantee D requires conflicts to be deterministically readable as conflicts through standard substrate access paths. Suppression in derivative views per A1.16 Pattern B may be admissible; suppression at substrate layer fails component (a).

5. Why Guarantee D is load-bearing for downstream commitments

Guarantee D is load-bearing for several CKS commitments. It is load-bearing for the integrating determinism contract from A1.10 — without conflict-preservation determinism, the contract has a content class (conflicts) where determinism does not apply. It is load-bearing for the conflict-as-first-class commitment from A1.03 and the substrate-level preservation from A2.13: without Guarantee D, A1.03's commitment is aspirational at the determinism level. It is load-bearing for cell-level resolution per A2.14 and the two-level coupling per A2.15: component (b) ensures substrate-level preservation remains intact across cell-level resolutions. It is load-bearing for the

substrate-as-source-of-truth commitment per A1.08, specifically for the source-of-truth-for-conflicts category per A2.45: A2.45 commits the substrate to authority for "what is in conflict," and Guarantee D is what makes that authority operationally realizable. It is load-bearing for path retraceability from A1.07: reconstruction that traverses conflicts and their resolutions needs the records deterministically readable per Guarantee A and addressable per Guarantee C.

6. Failure modes that violate the guarantee

Each anti-pattern below names a way an implementation can fail the architectural commitment.

(a) Conflict-erasure-on-resolution. The conflict state is erased when a resolution occurs, leaving only the resolved state in substrate. Component (b) fails; A1.03 and A2.13 are operationally violated.

(b) Conflict-summarization. Conflict records are periodically summarized into aggregate statistics or compressed representations that lose original conflict content; component (a) fails for summarized conflicts.

(c) Last-write-wins concurrency overrides. LWW concurrency control automatically resolves simultaneous-write conflicts by keeping only the latest write. The first-class-conflict commitment from A1.03 is violated by infrastructure layer; component (b) fails.

(d) Eventual-consistency conflict reconciliation. The substrate operates in eventually-consistent architecture where replicas reconcile conflicts through CRDT or similar mechanisms. Conflicts are reconciled away as part of normal infrastructure operation; preservation fails at the determinism level even when nominal at write time.

(e) Conflict-relationship variation across reads. Relationship metadata per A2.16 differs across reads of the same conflict — characterizations that vary by reader, time, or access path. Component (d) fails; A2.16's relationship-as-substrate-content commitment is operationally violated.

(f) Resolution-decision-overwrites-conflict. Resolution decisions are recorded by overwriting the conflict state in place rather than as additional substrate content per Guarantee C. The original conflict is no longer addressable; both component (b) and Guarantee C fail.

(g) Conflict-reads-via-derivation-only. Conflict reads are produced by deriving them at read time from other substrate state — for example, computing conflicts from current values across multiple entities. The derivation may vary across reads; component (a) fails for derived conflicts even when the underlying entities are deterministic.

(h) Conflict-provenance-stripping. Conflicts are recorded but the four conflict-specific fields per A2.16 or the six broader fields per A2.40 are stripped. Provenance is non-deterministic or absent; component (c) fails. Path retraceability per A1.07 cannot operate over the affected conflicts.

(i) Conflict-archival-external. Recent conflicts persist in substrate; older conflicts are archived to external systems outside the substrate's standard access paths. Component (a) fails for archived conflicts.

(j) Conflict-suppression-in-views. Conflicts are filtered out of standard read paths at substrate layer (not merely in derivative views per A1.16 Pattern B) to "improve user experience." Reads vary depending on which view is used at substrate layer; component (a) fails.

7. Operational test

A system satisfies Guarantee D if and only if all of the following are true at all times during the substrate's existence:

1. For any substrate state S containing conflict states preserved per A2.13, reads against S return the same conflict content (contradicting positions, relationship metadata per A2.16, conflict provenance per A2.40) deterministically across time, readers, access paths, and replicas.
2. Cell-level resolution decisions per A2.14 are recorded as additional substrate content per Guarantee C without erasing, overwriting, or modifying the original conflict state in place.
3. Conflict provenance — the four conflict-specific fields per A2.16 and the six broader fields per A2.40 — is deterministic across operations.
4. Conflict relationships are stable across operations that do not explicitly modify them; relationship reads return consistent metadata regardless of which side of the contradiction is queried, which access path is used, or which time within the persistence interval the read occurs.
5. Conflicts are not reconciled, suppressed, or erased by infrastructure-layer mechanisms (LWW concurrency control, CRDT reconciliation, eventual-consistency convergence, automatic merge logic); the first-class-conflict commitment from A1.03 is preserved at the determinism level, not only at write time.
6. Conflict reads are queryable through standard substrate read operations per A2.25 Requirement 2 and substrate-only access paths per A2.41, with the four determinism properties holding throughout the conflict's substrate residence.

A system that fails any of (1)–(6) does not satisfy Guarantee D in the architectural sense, even if it preserves conflicts in some operational sense.

8. Why naming Guarantee D as standalone matters

Implementations under pressure to support multi-user contexts or deliver smooth user experience drift toward conflict-erasure or conflict-suppression patterns. The drift is steady because conflict preservation is operationally more demanding than conflict resolution-and-erasure: preserved conflicts require interfaces that surface them, governance attention that addresses them, and explicit handling in cells that consume them. Erasure or suppression simplifies the operational picture at the cost of architectural commitment.

Implementations that drift away from Guarantee D produce systems where conflicts appear preserved nominally but vary across operations, are erased on resolution, or are suppressed in standard views. The downstream consequences manifest as conflict-as-first-class commitment failure (A1.03 becomes

operational-only without determinism rigor); source-of-truth fragmentation for the contradictions category (A2.45 fails because conflict authority is ambiguous when reads are non-deterministic); regression-testing failures (conflict-state assertions cannot be reliably verified); and audit-coherence failures (governance reviews cannot reliably retrieve historical conflict states).

Naming Guarantee D as standalone — with the four operational components, the limitations, the adjacent-pattern distinctions, the load-bearing connections, the failure modes, and the operational test specified above — gives downstream readers a precise specification of what conflict-preservation determinism the architecture requires. Subsequent notes A2.61 (source of truth), A2.62 (allowed non-determinism), and A2.63 (regression testing) complete the determinism contract's operational decomposition.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Guarantee D — Conflict States Are Preserved: Conflict-Preservation Determinism as a Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern*. May 5, 2026. ORCID: 0009-0004-8065-3235.